Rapport technique

Projet ML complet - Analyse de sentiment YouTube

YouTube Sentiment Lens

Du développement au déploiement cloud

Filière INDIA | Année universitaire 2026/27
Université Mohamed V - ENSAM Rabat
Prof. Maria Zemzami

01 | Synthèse et problématique

Objectif

Le projet automatise l'analyse de commentaires YouTube en trois classes : négatif (-1), neutre (0) et positif (1).

Problématique

L'analyse manuelle de centaines de commentaires est chronophage, subjective et difficile à quantifier. Le système fournit une analyse automatique, rapide et reproductible.

Solution

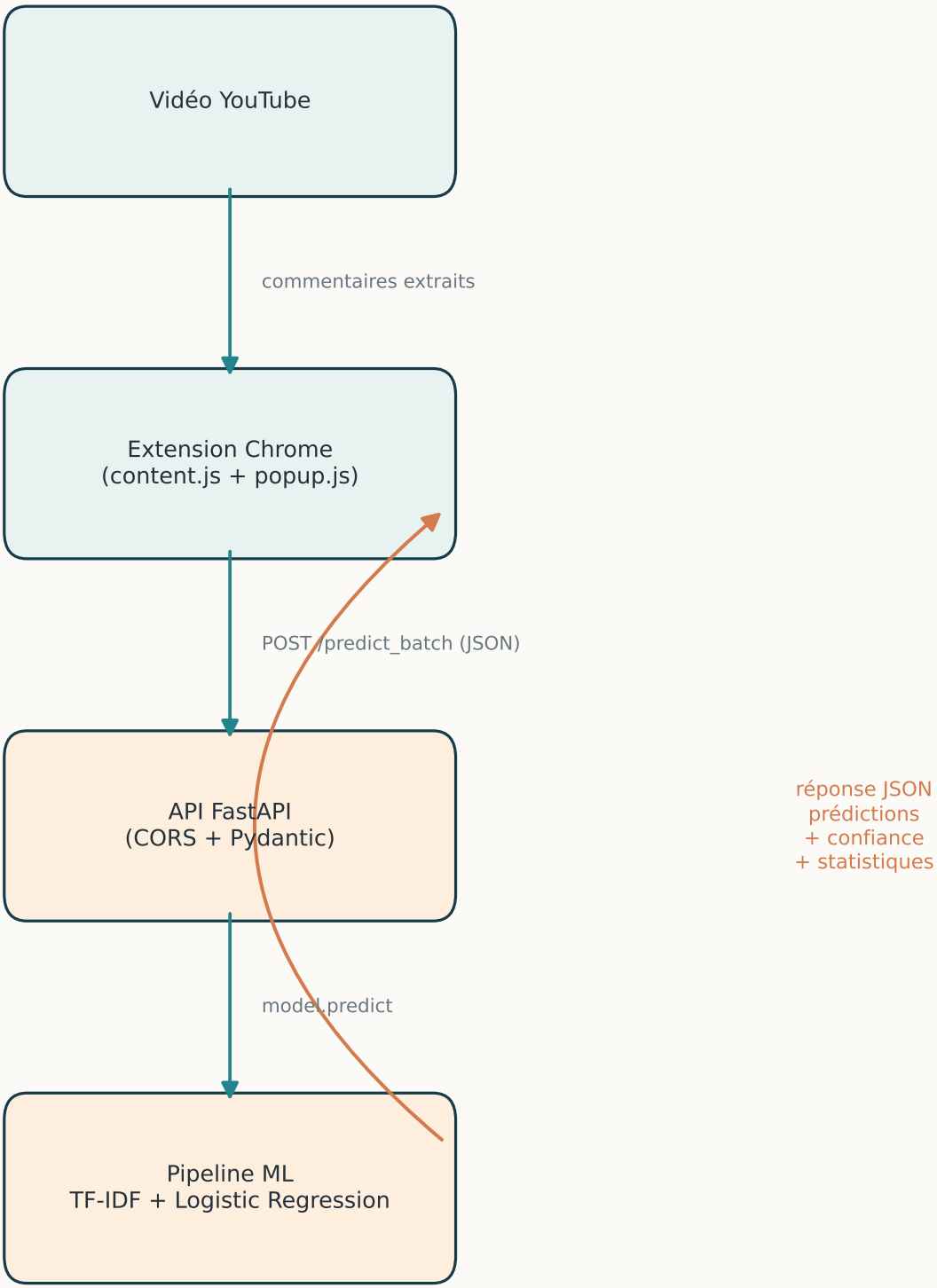
Une extension Chrome extrait les commentaires visibles, les transmet par batch à une API FastAPI, puis affiche le sentiment, la confiance, un graphique de répartition et des filtres.

Résultat

Le modèle atteint 87,79 % d'accuracy et dépasse tous les seuils F1 demandés. L'API est servie dans un conteneur Docker exécuté sur GitHub Codespaces.

03 | Architecture du système

Flux de données de bout en bout



Déploiement : conteneur Docker sur GitHub Codespaces, port 7860 rendu public (HTTPS).

04 | Données et préparation

Source

Le dataset Reddit Sentiment Analysis est téléchargé automatiquement depuis GitHub et sauvegardé dans data/raw/reddit.csv. Il contient 37 249 commentaires (brut : négatif 8 277, neutre 13 142, positif 15 830).

Nettoyage

Le texte est mis en minuscules; les URLs, mentions et hashtags sont remplacés par des tokens (URL, USER, HASHTAG); les caractères spéciaux sont normalisés; les doublons et les textes de moins de trois caractères sont supprimés.

Statistiques

Après nettoyage, 36 562 commentaires sont conservés (négatif 8 233, neutre 12 606, positif 15 723), soit bien plus de 1 000 exemples par classe comme exigé.

Reproductibilité

Le split stratifié (random_state=42) répartit 80 % en entraînement (29 249) et 20 % en test (7 313), ce qui préserve la distribution des classes.

05 | Modélisation et optimisation

Vectorisation

TF-IDF (avec sublinear_tf) valorise les termes discriminants et réduit le poids des mots trop fréquents, ce qui convient mieux qu'un simple comptage.

Comparaison des algorithmes

Deux pipelines pondérés par classe ont été comparés : Logistic Regression (accuracy brute 86,18 %) et Linear SVM (89,89 %).

Choix du modèle

Le Linear SVM est légèrement meilleur en accuracy brute, mais la Logistic Regression a été retenue car elle fournit des probabilités (predict_proba), indispensables au score de confiance de l'extension.

Optimisation

GridSearchCV (cv=3, score f1_macro) explore C, min_df et ngram_range. Meilleurs paramètres : C=2.0, min_df=2, ngram_range=(1, 1).

Limite méthodologique

La sélection entre candidats s'appuie sur le jeu de test; un jeu de validation dédié serait plus rigoureux et constitue une amélioration future.

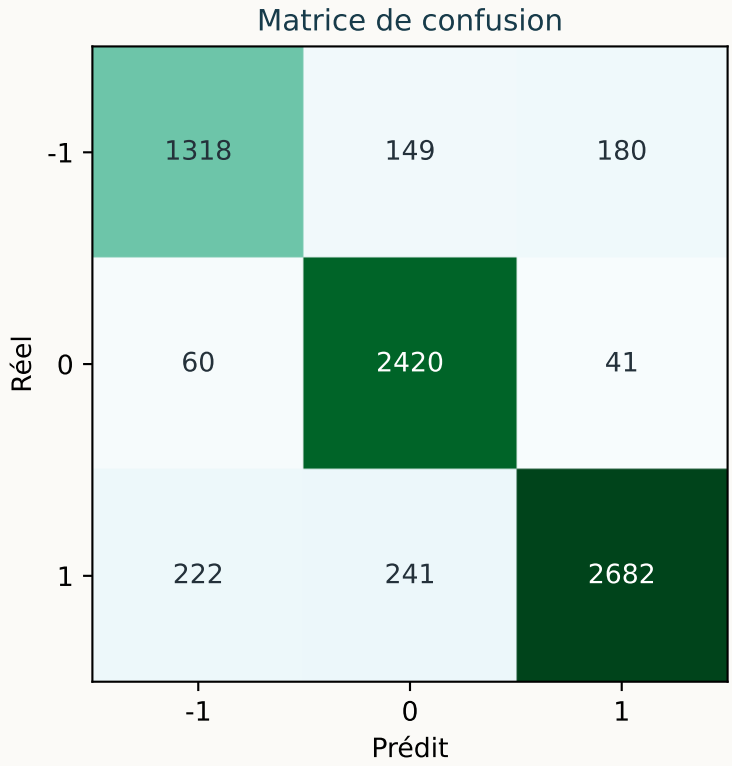
Artefacts

Le pipeline, le vectoriseur et le classifieur sont sauvegardés avec joblib dans models/.

06 | Performance du modèle

Jeu de test : 7 313 commentaires

Mesure	Résultat	Seuil	Statut
Accuracy	87,79 %	80 %	Atteint
F1 négatif	81,18 %	75 %	Atteint
F1 neutre	90,79 %	75 %	Atteint
F1 positif	88,69 %	75 %	Atteint
Inférence batch 50	4,08 ms	100 ms	Atteint



07 | API REST et extension Chrome

API

GET /health vérifie le service et le chargement du modèle. POST /predict_batch valide un batch de 1 à 500 commentaires via Pydantic et renvoie, par commentaire, le label, le sentiment, la confiance, puis les statistiques globales et la latence.

Robustesse

CORS est activé pour l'extension; un batch vide, trop grand, mal typé ou contenant un commentaire vide renvoie une erreur 422; un modèle absent renvoie 503.

Extension

L'extension Manifest V3 extrait les commentaires visibles, affiche les pourcentages, un graphique à barres de répartition, filtre par sentiment, propose un mode clair/sombre, un indicateur de chargement et la copie des résultats.

08 | Déploiement cloud

Conteneurisation

Le Dockerfile part de python:3.10-slim, installe uniquement requirements-prod.txt, copie le code et les modèles, et expose un port configurable via la variable PORT (défaut 7860).

Exécution sur Codespaces

Commandes : docker build -t youtube-sentiment-api . puis docker run -d -p 7860:7860 -e PORT=7860 youtube-sentiment-api. Le port 7860 est ensuite rendu public dans l'onglet Ports.

Contrainte

L'URL Codespaces (https://<nom>-7860.app.github.dev) est éphémère : le Codespace s'arrête après environ 30 min d'inactivité et doit être relancé avant la démonstration. C'est la différence clé avec un hébergement PaaS permanent.

Validation

Les endpoints /health, /docs et /predict_batch ont été vérifiés sur l'URL publique, ainsi que le flux complet extension -> API cloud -> résultats.

09 | Difficultés, tests et améliorations

Difficultés rencontrées

GridSearchCV échouait avec BrokenProcessPool sous Windows : le parallélisme a été fixé à n_jobs=1. L'interpréteur de VS Code ne pointait pas vers l'environnement contenant les dépendances : il a été reconfiguré.

Tests

La suite pytest couvre l'API (health, batch vide, hors limite, payload invalide, prédiction cohérente) et le modèle (accuracy >= 80 %, F1 par classe > 75 %, latence < 100 ms, cas limites : texte court/long, emojis, langues mélangées).

Améliorations futures

Jeu de validation dédié pour la sélection de modèle; authentification de l'API; monitoring de dérive; mise à jour versionnée du modèle; extraction des réponses (threads) et défilement automatique côté extension; cache des résultats.

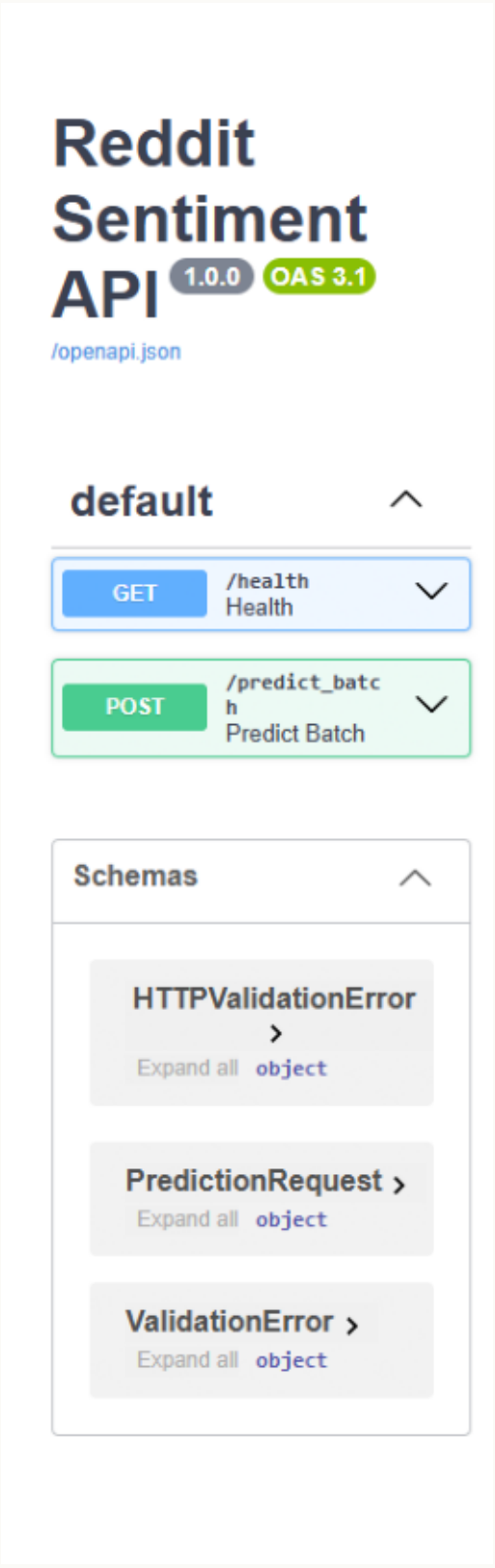
10 | Preuves - API

Preuve capturée avec Playwright

GET /health



Swagger /docs



11 | Preuves - Extension

Preuve capturée avec Playwright

Mode clair



Mode sombre

